

Praxis: I²C-/SPI-Treiber erweitern

Vom Startercode zur nachvollziehbar getesteten Änderung

4.08

Drei Fragen führen durch das Abschlusslabor

- | | | | | | |
|-----------|--|-----------|--|-----------|--|
| 01 | Was wird genau geändert? | 02 | Welche Evidenz belegt die Funktion? | 03 | Wie bleibt der Versuch sicher? |
| | Ein begrenzter Gerätepfad und sein beobachtbares Verhalten bilden den Scope. | | Build, Binding, Rückgabewerte und Log müssen dieselbe Aussage stützen. | | Definierte Hardware, kontrollierte Fehler und vollständiger Rückbau setzen die Grenze. |

Lernziel

Am Ende kannst du einen vorbereiteten Bustreiber erweitern, reproduzierbar testen und technisch erklären.

Vier Schritte machen aus Code eine überprüfte Änderung

Verstehen

Geräteprotokoll, Callback, Kontext und Ausgangszustand klären.

Implementieren

Die kleinste Änderung mit vorhandenen Kernel-APIs umsetzen.

Bauen

Modul, Warnungen und Metadaten gegen den Zielkernel prüfen.

Verifizieren

Binding, Funktion, Fehlerfall und Rückbau reproduzierbar belegen.

LABORPFAD

Ablauf: Code → Build → Bind → Test → Remove

Evidenz: Diff → Modul → sysfs → Kernel-Log

Grenze: vorbereitete Hardware oder Emulator

Der Startertreiber enthält den vollständigen Lebenszyklus

STARTERSTRUKTUR

```
match table
probe()
struct demo_state
regmap
subsystem callback
remove() / devres
module_*_driver()
```

01

Binding

Match-Tabelle und Bus-Core rufen probe() für das richtige Gerät auf.

02

Zustand

Ein gerätespezifisches Objekt hält Regmap, Sperren und Laufzeitdaten.

03

Datenpfad

Ein bestehender Subsystem-Callback übersetzt Register in Kernelwerte.

Das Labor braucht eine eindeutig definierte Zielumgebung

LABORRAHMEN

```
Kernelversion + .config  
passende Header  
Startermodul  
festgelegtes Busgerät  
Datenblatt / Emulator  
serielle Konsole
```

01

Baseline

Boot, vorhandenes Binding und Kernel-Log ohne eigene Änderung sichern.

02

Ziel

Erwartetes Register, Ergebnis und sichtbare Schnittstelle notieren.

03

Grenze

Nur freigegebene Laborhardware oder einen vorbereiteten Emulator verändern.

Vor der Änderung muss der Ausgangszustand belegt sein

BASELINE

```
Businstanz  
Geräteadresse / Chip Select  
modalias  
Driver-Link  
compatible / IDs  
relevante Logzeilen
```

Zwei Sichtweisen

Software

Modulmetadaten, Match-Tabelle und Treiberlink müssen zusammenpassen.

Hardware

Bus, Adresse, Mode, Takt und Versorgung müssen der Zieldefinition entsprechen.

Erst dokumentieren, dann ändern: Die Baseline ist der Vergleichspunkt für jeden Test.

Managed State und Regmap bilden die Grundlage

PROBE-CODE

```
st = devm_kzalloc(dev, sizeof(*st), GFP_KERNEL);
if (!st)
    return -ENOMEM;
i2c_set_clientdata(client, st);
st->map = devm_regmap_init_i2c(client, &cfg);
if (IS_ERR(st->map))
    return dev_err_probe(dev, PTR_ERR(st->map),
        "regmap\n");
```

Lies als Vertrag

01 Ownership

devm_* bindet Speicher und Regmap an die Device-Lebensdauer.

02 Zugriff

Regmap kapselt Registerformat, Endianness und Busoperationen.

03 Fehler

Der echte errno-Wert bleibt erhalten und erhält Device-Kontext.

Für SPI die bus-spezifischen Device- und Regmap-Aufrufe austauschen; Zustands- und Fehlervertrag bleiben gleich.

Die Geräteidentität ist der erste kontrollierte Transfer

IDENTITÄT

```
ret = regmap_read(st->map, REG_CHIP_ID, &id);
if (ret)
    return dev_err_probe(dev, ret, "chip id\n");
if (id != EXPECTED_ID)
    return dev_err_probe(dev, -ENODEV,
                        "unexpected id: %#x\n", id);
```

Drei Entscheidungen

01

Adresse

Register und Breite stammen aus Datenblatt oder Emulatorvertrag.

02

Transport

Regmap liefert Busfehler unverändert an den Treiber zurück.

03

Semantik

Eine unerwartete ID verhindert das Binding mit -ENODEV.

Ein vorhandener Subsystem-Callback wird gezielt ergänzt

DATENPFAD

```
ret = regmap_read(st->map, REG_STATUS, &val);  
if (ret)  
    return ret;  
  
*active = !(val & STATUS_READY);  
return 0;
```

Vom Register zum Kernelwert

01

Vertrag

Signatur, Einheit und Fehlersemantik des Subsystems beibehalten.

02

Umwandlung

Bitmaske und Skalierung explizit aus dem Geräteformat ableiten.

03

Synchronisation

Sperre nur ergänzen, wenn mehrere Pfade denselben Zustand teilen.

Fehlerpfad und Rückbau spiegeln den Aufbau

RÜCKBAU

```
probe()
```

```
    Versorgung aktivieren
```

```
    Hardware initialisieren
```

```
    IRQ / Work starten
```

```
remove()
```

```
    neue Zugriffe sperren
```

```
    Work synchron stoppen
```

```
    Hardware deaktivieren
```

Zwei Ebenen

Managed

devm_* gibt Ressourcen in umgekehrter Reihenfolge frei.

Explizit

Aktive Arbeit, IRQ-Quellen und Hardwarezustand bewusst beenden.

Ein erfolgreicher Unload- und Reload-Test deckt viele Lebensdauerfehler früh auf.

Build und Deployment müssen zum laufenden Kernel passen

BUILD & LOAD

```
$ make M=$PWD W=1 modules
$ modinfo ./demo_bus.ko
$ sudo insmod ./demo_bus.ko
$ dmesg | tail -n 80
$ sudo rmmod demo_bus
```

Der kontrollierte Zyklus

01

Bauen

Warnungen, vermagic und Modulabhängigkeiten vor dem Laden prüfen.

02

Binden

Driver-Link und erwartete Probe-Meldung nach dem Laden belegen.

03

Entfernen

Modul entladen und Log auf Warnungen oder hängende Arbeit prüfen.

Bei WARN, OOPS oder unerwartetem Hardwarezustand den Versuch stoppen und die Konsole sichern.

Drei Tests trennen Funktion von Zufall

TESTMATRIX

Positivtest

erwartetes Gerät + gültiger Wert

Negativtest

vorbereiteter Busfehler oder falsche ID

Wiederholung

load → read → unload × 5

Regression

bestehender Callback bleibt stabil

Drei Evidenzklassen

01 Funktion

Ausgabewert und Registerbedeutung stimmen reproduzierbar überein.

02 Fehler

Erwarteter errno-Wert und verständliche Meldung erscheinen.

03 Lebensdauer

Wiederholtes Binden und Entfernen bleibt frei von Warnungen.

Praxis: Evidenz für die Treiberänderung sammeln

TERMINAL

```
$ git diff --check
$ make M=$PWD W=1 modules
$ modinfo ./demo_bus.ko
$ readlink -f /sys/bus/*/devices/*/driver
$ dmesg | tail -n 80
```

Belege drei Aussagen

01

Quelle

Diff zeigt eine kleine Änderung ohne Whitespace-Fehler.

02

Artefakt

Modul wurde für den Zielkernel gebaut und trägt passende Metadaten.

03

Laufzeit

Driver-Link, Funktionswert und Log zeigen denselben Gerätepfad.

Befehle an den konkreten Buspfad anpassen; breite Wildcards nur read-only verwenden.

Praxis: Treiberänderung bauen, testen und erklären

01 Scope wählen

Eine fehlende Registerauswertung oder Validierung im Startercode wählen.

02 Pfad ergänzen

Regmap-Zugriff und vorhandenen Subsystem-Callback vervollständigen.

03 Fehler prüfen

Busfehler und unerwartete Geräteidentität korrekt weitergeben.

04 Zyklus testen

Build, Load, Funktionsprüfung und Unload reproduzierbar ausführen.

05 Evidenz sichern

Diff, Befehle, Ergebnis und relevante Logzeilen dokumentieren.

06 Peer Review

Partner prüft Kontext, Ownership, Rückbau und Behauptungen.

Nur vorbereitete Laborziele verändern; bei Kernelwarnungen sofort stoppen.

Fünf Tage verbinden
Kontext, Ownership und Evidenz.

Formuliere in einem Satz:

Kannst du den Weg vom Boot über Task und Speicher bis zum gebundenen Gerät erklären?

Abschluss: beobachten → begründen → sicher verändern